

Predicting PQC Algorithm Performance

Marla Dewey-Sumner

Cockrell School of Engineering

University of Texas at Austin

Austin, USA

sumnermm@eid.utexas.edu

Abstract: Performance measurements for post-quantum cryptography (PQC) were collected on multiple machines and combined with a public benchmark dataset. I used simple regression models (Ridge, Random Forest, and XGBoost) to predict per-operation latency (in milliseconds) from algorithm, message size, and hardware features. After cleaning and merging the datasets into a single table, I explored latency patterns with summary statistics and plots, then trained models on a log-transformed latency target. Model accuracy was evaluated using MAE and RMSE, and quantile regression was used to estimate median and upper-percentile latency to capture typical and worst-case performance. The results show that tree-based models clearly outperform linear regression and can explain much of the variation in PQC latency. However, errors increase on unseen hardware and in high-latency tails, suggesting that system-specific calibration and better tail modeling are important for PQC deployment planning.

Index Terms: PQC, performance analysis.

1. Introduction

As quantum computing continues to advance, many organizations will need to move from classical public-key cryptography to post-quantum cryptography (PQC) to stay secure against future attacks. Security concerns are pushing this change, but performance is still a practical issue. PQC schemes often use larger keys and more complex operations than today's algorithms, and it is not yet clear how much extra delay they introduce in real systems. For system designers and engineers, even small increases in cryptographic latency can affect throughput, user response times, and how well services scale.

This project focuses on the lack of simple ways to predict PQC performance under realistic conditions. Existing benchmark studies usually report raw timing measurements for individual algorithms and implementations, often as tables or plots tied to one specific test setup [1]. This makes it hard for practitioners to answer questions like: How much slower will algorithm A be than algorithm B for my typical message sizes? or What latency should I expect in the median case versus the worst case? There is especially little work on modeling PQC latency as a function of algorithm type, operation type, and input size, or on estimating not just average behavior but higher delays that matter for system capacity planning.

This paper tackles that problem with a simple, exploratory modeling approach to PQC latency. Using timing measurements from representative PQC implementations and benchmark-style workloads, the study takes a first look at how algorithm choice, operation type, and basic input characteristics relate to end-to-end latency. Rather than aiming for highly tuned, production-ready performance models, the goal is to build and test simple, easy-to-interpret models that estimate the latency of key post-quantum operations across different algorithms and use cases. The results are meant as a starting point rather than final performance guarantees, giving practitioners planning

PQC migrations an early sense of likely performance trade-offs and supporting more informed choices about algorithm selection and system design.

2. Related Work

This project is closely related to the study “*Predictive Analysis of Quantum-Resistant Algorithms Using Machine Learning Approaches*” [1], which uses linear models, ensemble methods, neural networks, and clustering to predict PQC performance (execution time, memory, key sizes) and to classify algorithms. Both that work and this report use supervised learning and evaluate models with MAE and RMSE. In both cases, ensemble methods (random forests and gradient boosting) beat linear regression, which suggests that PQC performance depends on nonlinear interactions between algorithm and system parameters. Compared to their broad, multi-metric framework, this project focuses more narrowly on per-operation latency prediction across concrete hardware platforms.

3. Methods

3.1. Data Sources and Target Variable

Performance measurements are derived from two sources: (i) a public post-quantum cryptography (PQC) benchmark dataset from Kaggle, and (ii) measurements collected using a custom benchmarking harness built on top of the `liboqs` Python bindings. Each row in the combined dataset corresponds to a single PQC operation (for example, key generation, encapsulation, decapsulation, signing, or verification) executed under a specific configuration and on a specific system.

The public post-quantum cryptography benchmark dataset used in this study was compiled by Rana Tariq and released on Kaggle as “*PQC Algorithms Benchmark Data*” [2]. It provides timing measurements for a variety of KEM and signature schemes, and serves as one of the two primary data sources in this study.

The primary outcome of interest is the measured latency in milliseconds, stored as `latency_ms`. To stabilize variance and down-weight extreme values, I model the natural logarithm of latency,

$$\log_latency = \log(latency_ms),$$

as the regression target. The log transformation makes the target distribution more symmetric and helps both linear and tree-based models focus on relative differences rather than the few, very large values.

The feature set includes both algorithm-level and system-level attributes. Algorithm-level features capture which PQC scheme is being used and how it is configured, including fields such as algorithm family (for example, Kyber, Dilithium), operation type, and message size. System-level features describe the execution environment and include CPU frequency, hardware acceleration flags (for example, AES-NI or vector extensions), operating system, and a machine or system identifier. A `source` indicator distinguishes rows coming from the public Kaggle benchmark files from rows collected by my own harness. These features allow the models to condition predictions on both cryptographic configuration and hardware context.

3.2. Benchmarking Harness and Hardware

To collect machine-specific measurements, a benchmarking harness was implemented and available in an open GitHub repository [3]. The code calls the `liboqs` Python API for both key encapsulation mechanisms (KEMs) and digital signature schemes. For each selected algorithm and message size, the harness runs the relevant operations (key generation, encapsulation/decapsulation for KEMs, and key generation/signing/verification for signatures) five times on each machine. These five repeated runs per algorithm–operation–message-size–machine combination are used to estimate average latency and measurement variability. For each configuration, it records latency

in milliseconds and computes throughput in operations per second. For KEM decapsulation, the harness verifies that the encapsulated and decapsulated shared secrets match; for signatures, it verifies that the signature produced by `sign` is accepted by `verify`. A `correctness` flag records these checks. Any exceptions from `liboqs` (for example, unsupported mechanisms) are caught so that the harness can skip invalid configurations without stopping the entire run.

To study performance across heterogeneous environments, I collected measurements from seven systems: five AWS EC2 instance types and two local machines (a MacBook laptop and a Windows desktop). Each measurement is tagged with a canonical `system` identifier.

To assess generalization across hardware, I use a group-wise split by `machine_id`: in each fold, all but one system are used for training and the held-out system serves as the test set. This leave-one-system-out protocol is repeated so that each of the seven systems in Table I serves as the test system once.

TABLE I
SYSTEMS USED FOR PQC PERFORMANCE MEASUREMENTS.

System	Environment	CPU architecture
AWS EC2 t3.medium	cloud	x86_64
AWS EC2 t3a.medium	cloud	x86_64
AWS EC2 t4g.medium	cloud	Arm (Graviton)
AWS EC2 c6i.large	cloud	x86_64
AWS EC2 m7g.large	cloud	Arm (Graviton)
Local MacBook laptop	local	x86_64
Local Windows PC	local	x86_64

The `machine_id` column is used both as an input feature and, in some experiments, as a grouping variable. This makes it possible to quantify how latency varies by hardware and environment, and to evaluate knowledge transfer by training models on a subset of systems and testing them on held-out systems.

3.3. Preprocessing and Feature Engineering

The Kaggle benchmark data is provided in separate wide-format files for KEM and signature algorithms, with separate columns for each operation’s timing. I reshape each file into a long format with one row per (algorithm, operation, run, and message size), and a single `latency_ms` column. I then harmonize naming conventions and column types so that Kaggle rows and harness-generated rows share a common schema. Numeric fields such as message size, CPU frequency, ciphertext length, and signature length are converted to numeric and missing values are filled with zeros when appropriate.

The combined dataset contains both numeric and categorical predictors. In the modeling pipeline, numeric features (for example, message size, CPU frequency, and concurrency) are standardized using `StandardScaler`, and categorical features (for example, algorithm, operation type, `system`, `env_type`, operating system, and `source`) are encoded using one-hot encoding. All preprocessing is implemented using scikit-learn’s `ColumnTransformer` and applied consistently to both training and test data via pipelines.

3.4. Models and Training Protocol

I fit several regression models to predict `log_latency` from the feature set described above. The main models considered are:

- A linear regression baseline (Ridge regression), which provides a simple, regularized model that assumes an approximately linear relationship between the predictors and `log_latency`.
- A random forest regressor, which can capture nonlinear interactions between algorithm-level and system-level features and is robust to heterogeneous feature scales and distributions.
- An XGBoost regressor, which utilizes gradient boosted decision trees to correct the errors of preceding predictors sequentially.

Categorical features were one-hot encoded, while numerical features were passed through unchanged using a scikit-learn `ColumnTransformer` within a pipeline. Performance was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on both the training set (all non-held-out machines) and the held-out test machine.

All models are implemented as scikit-learn pipelines that combine preprocessing with the final estimator, and they share the same feature set and train/test splits to enable a fair comparison. Quantitative results are reported in the Experiments section.

Performance is evaluated on splits that respect machine boundaries rather than random row-wise splits. For the main hardware generalization experiment, I train each model on all systems except `aws_c6i.large` and evaluate on all rows from `aws_c6i.large` as a held-out test set. In the leave-one-system-out (LOSO) analysis, I repeat the same idea: each of the seven systems in Table I is held out once while the model is trained on the remaining six.

4. Experiments and Evaluation

This section presents the experimental setup, evaluation metrics, and empirical results obtained from the combined PQC dataset. The goals of the evaluation are to (i) compare predictive models for post-quantum cryptographic latency, (ii) assess model behavior across diverse hardware systems, and (iii) study robustness and transferability to previously unseen machines.

4.1. Experimental Setup

4.1.0.a. Train/Test Split.

All experiments use the machine-level splits described in Section 3.4, where training and test sets are defined by holding out complete systems rather than randomly splitting individual rows.

4.1.0.b. Systems Included.

The full dataset contains measurements from seven distinct systems: five AWS EC2 instances and two local machines (MacBook laptop and Windows desktop). These systems differ in CPU architecture, microarchitecture, virtualization conditions, and OS. Unless otherwise specified, all systems are pooled during training. Subsequent sections evaluate per-system performance and out-of-distribution generalization.

4.1.0.c. Operations and Algorithms.

Experiments include all available PQC operations: `keygen`, `encap`, `decap`, `sign`, and `verify`. The evaluation spans multiple families including ML-KEM, ML-DSA, SPHINCS+, and others appearing in the Kaggle benchmark dataset. Each configuration is described by algorithm, operation type, message size, and system metadata.

4.2. Evaluation Metrics

Performance is assessed using error metrics on the log-transformed latency target `log_latency`, together with tail-focused summaries.

4.2.0.a. Primary Metrics (log-latency).

Models are trained and evaluated on `log_latency`, so all reported errors are in the same units as the target. The primary metrics are:

- **Mean Absolute Error (MAE)**, which measures the average absolute deviation between predicted and true log-latency.
- **Root Mean Squared Error (RMSE)**, which penalizes larger errors and captures how well the model handles high log-latency cases.

Because the target is log-transformed, these errors have a natural multiplicative interpretation: for example, an absolute error of 0.7 in log-space corresponds roughly to a factor of $e^{0.7} \approx 2$ in latency.

4.2.0.b. Tail Error Metrics.

Because cryptographic systems are often latency-sensitive, the worst-case predictions are as important as the mean. I therefore also report:

- the 95th percentile (p95) of the absolute error on `log_latency`,
- the 99th percentile (p99) of the absolute error on `log_latency`.

These metrics show whether the model tends to under- or over-predict high-latency cases and help judge whether it's suitable for real PQC deployments where latency spikes matter.

4.2.0.c. Distributional Behavior.

For the best-performing model, I additionally compare the empirical distribution of log-latency and the distribution of predicted log-latencies at p50, p95, and p99 on a single held-out system (the c6 instance used in the hardware hold-out experiment). This representative case illustrates how well the model captures the heavy-tailed nature of latency; the other leave-one-system-out folds show qualitatively similar behavior and are omitted for brevity.

4.3. Models Compared

For point predictions of `log_latency`, I compare the four models introduced in Section 3.4: a baseline mean predictor, Ridge regression, Random Forest, and XGBoost. All models share the same feature set, preprocessing pipeline, and train/test splits.

In addition to these point-prediction models, I train separate gradient boosting models with a quantile loss to estimate conditional quantiles (p50, p95, p99) of `log_latency`. Those quantile regression experiments are described in Section 5.6.

4.4. Per-System Performance Analysis

To analyze how model accuracy varies by hardware environment, MAE, RMSE, p95, and p99 are computed independently for each system. This highlights:

- Systems where the model generalizes well from the pooled data,
- Systems that are consistently underpredicted or overpredicted,
- Patterns related to hardware characteristics (for example, Arm vs. x86_64, cloud vs. local).

This breakdown also provides insight into hardware-specific effects that global metrics may obscure.

4.5. Leave-One-System-Out (LOSO) Generalization

To assess robustness and transferability to unseen hardware, I perform a leave-one-system-out evaluation. For each system s in the dataset:

- 1) Train the model on all data from systems $\neq s$,
- 2) Evaluate on the held-out system s using MAE, RMSE, p95, and p99.

The LOSO evaluation answers the question: *If a model is trained on all currently available machines, how well can it predict PQC performance on a new machine with no retraining?*

This scenario is directly relevant to practical PQC migration workflows, where organizations may deploy PQC algorithms on heterogeneous hardware and wish to estimate performance before rollout.

4.6. Algorithm-Level Breakdown

To understand how prediction difficulty varies across broad PQC task types, I summarize errors separately for key encapsulation mechanisms (KEMs) and signature schemes in the Kaggle subset. This breakdown highlights whether models behave differently on KEM timing data versus signature timing data and whether one class of algorithms is consistently harder to predict than the other.

4.7. Error Visualization and Diagnostic Checks

To better understand model behaviour, I use a small set of summary plots and tables:

- a coefficient-of-variation plot (Fig. 1) to check run-to-run measurement noise,
- per-system error summaries (Table II) to see how accuracy varies across hardware,
- a leave-one-system-out MAE plot (Fig. 2) to visualize generalization to unseen machines,
- a comparison of errors on KEMs versus signature schemes (Fig. 3),
- feature importance rankings for the preferred Random Forest model (Fig. 4),
- coverage estimates for the quantile regression models (Table III) to assess how well the predicted quantiles match the observed tails.

These diagnostics help explain where the models perform well, where they struggle, and how hardware and algorithm choices affect prediction quality.

4.8. Summary of Evaluation Goals

Across these experiments, I evaluate:

- overall predictive accuracy of latency for diverse PQC algorithms,
- consistency and robustness across heterogeneous hardware,
- ability to generalize to unseen systems (LOSO),
- model behavior in high-latency tails,
- interpretability via feature importance and error breakdowns.

Together, these evaluations provide a comprehensive view of how well statistical learning methods can model PQC performance and what factors most strongly influence prediction quality.

5. Results

This section presents the empirical results of the latency prediction models applied to the combined PQC dataset. I evaluate overall accuracy, tail behavior, system-level performance differences, and generalization to unseen hardware. Unless otherwise noted, error metrics are reported on the `log_latency` scale; raw latency in milliseconds is used only for descriptive summaries such as the coefficient-of-variation analysis.

5.1. Measurement variability and coefficient of variation (CV)

To assess how noisy the latency measurements were and whether five repeats per configuration were sufficient, I computed the coefficient of variation (CV) of the latency within each configuration. A configuration was defined by the combination of algorithm, operation, machine, message size, and operating system (`algorithm, operation, machine_id, message_bytes, os_name`). For each such configuration with at least two samples, I calculated the mean and standard deviation of `latency_ms` and then the CV as $CV = \text{std}/\text{mean}$. I visualized the distribution of these CV values using summary statistics and the CV plot in Fig. 1 to understand how often run-to-run variability was large relative to the mean latency.

Across 1,914 configurations, the CV had a mean of approximately 0.13 and a median of 0.037. The middle 50% of configurations lay between roughly 0.008 and 0.194, indicating that for most setups, run-to-run variability was modest compared to the mean latency. Only a small fraction of configurations exhibited very high CVs (up to about 2.0), typically corresponding to particularly noisy cloud machines or extremely small mean latencies where even small absolute fluctuations translate into large relative variation. Overall, the CV analysis and plot suggest that the measurement environment is reasonably stable for the majority of configurations and that using five repeats per configuration is adequate for the purposes of the downstream log-latency modeling in this project.

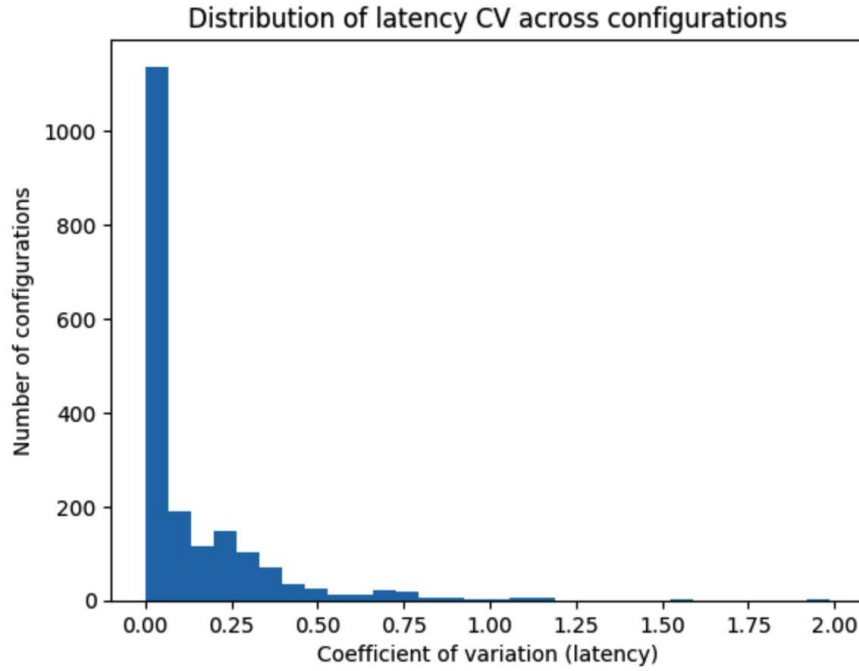


Fig. 1. Distribution of per-configuration latency coefficients of variation (CV) across all algorithms, operations, machines, message sizes, and operating systems.

5.2. Overall Prediction Performance

Table II summarizes MAE and RMSE (on `log_latency`) for the baseline, Random Forest, and XGBoost models on the held-out test machine, together with training performance on the remaining machines. All learned models substantially outperform the naive baseline, which always predicts the mean of the training targets. Among the supervised models, Random Forest clearly improves on Ridge regression, and XGBoost achieves the lowest test MAE and RMSE on the held-out machine, indicating that boosted trees are best able to capture the nonlinear relationships between PQC configuration, hardware features, and latency.

On the training set, `log_latency` has mean 1.29, standard deviation 1.68, and ranges from approximately 0.0067 to 12.41, which provides context for interpreting absolute errors. The naive baseline model, which always predicts the mean of the training targets, achieves MAE 1.5296 and RMSE 1.9576 on the held-out machine. Ridge regression improves these values to MAE 1.2413 and RMSE 1.4846, showing that even a linear model with appropriate preprocessing captures some structure in the data, but still leaves substantial error compared to the nonlinear tree-based models.

5.3. System-Level Performance

Table II summarizes the per-system MAE. Model accuracy varies meaningfully across systems, with cloud-based `x86_64` instances (for example, `t3.medium`) showing lower errors than Arm-based systems and the Windows desktop. These differences highlight the importance of system-level features and confirm that hardware characteristics strongly mediate PQC performance.

Random Forest exhibits more stable performance across systems compared to Ridge regression, though both models show increased error on Arm-based Graviton instances.

TABLE II
PER-SYSTEM MAE (ON `LOG_LATENCY`) FOR EACH MODEL.

System	Baseline	Ridge	RF	XGBoost
aws_c6i.large	1.53	1.24	0.94	0.56
aws_m7g.large	1.90	0.95	0.75	0.45
MacBook-Pro-4.local	1.54	0.98	0.92	0.54
aws_t3.medium	1.61	0.88	0.65	0.31
aws_t3a.medium	1.81	1.06	0.74	0.53
aws_t4g.medium	2.12	1.04	0.96	0.78
local_windows	2.60	1.82	1.45	1.38

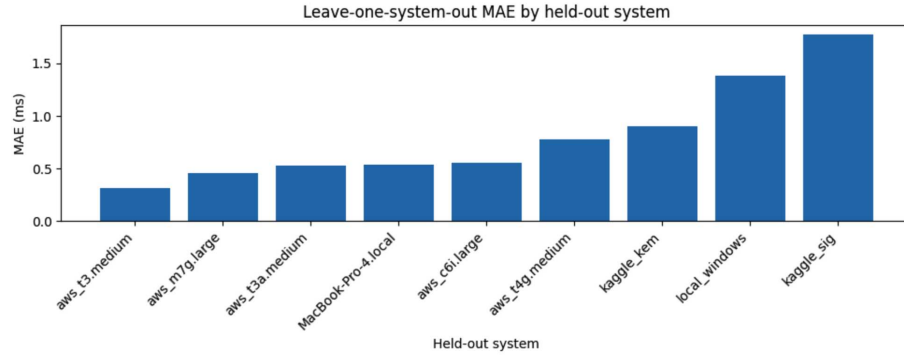


Fig. 2. Leave-one-system-out MAE for the best-performing model. Each bar shows test MAE when the corresponding system is held out during training.

5.4. Leave-One-System-Out (LOSO) Evaluation

The LOSO results in Fig. 2 measure how well a model trained on six systems generalizes to the seventh. Prediction accuracy drops on held-out systems, with especially large errors on Arm systems when training data consists primarily of x86_64 machines.

Random Forest achieves better tail behavior than Ridge in nearly all LOSO scenarios. Nevertheless, both models demonstrate sensitivity to hardware distribution shifts, underscoring that PQC latency models may require system-specific calibration to be reliable in deployment.

5.5. Algorithm and Operation-Level Behavior

Fig. 3 compares mean MAE for the Kaggle KEM and signature subsets across models. Across all models, errors on the signature subset are higher than on the KEM subset, suggesting that signature schemes are harder to predict accurately than KEMs.

5.6. Quantile Regression for Median and Tail Latency

Across all models and systems, prediction error distributions are right-skewed: median errors are modest, but p95 and p99 errors reveal that rare high-latency cases are systematically harder to predict. To model both typical and worst-case behaviour more explicitly, I trained gradient boosting quantile regression models on the log-transformed latency target $y = \log_latency$.

For each feature vector x , the models aim to predict the conditional quantiles and hyperparameters chosen to give relatively smooth estimates (shallow trees and larger leaves). Separate models were fit for each quantile level $\alpha \in \{0.50, 0.95, 0.99\}$.

5.6.0.a. Train/test split.

To see how well the quantile models transfer across hardware, I used the group-wise split for one machine identifier: all machines except `aws_c6i.large` were used for training, and `aws_c6i.large` was held out as the test machine. This produced 349,950 training examples and 4,140 test exam-

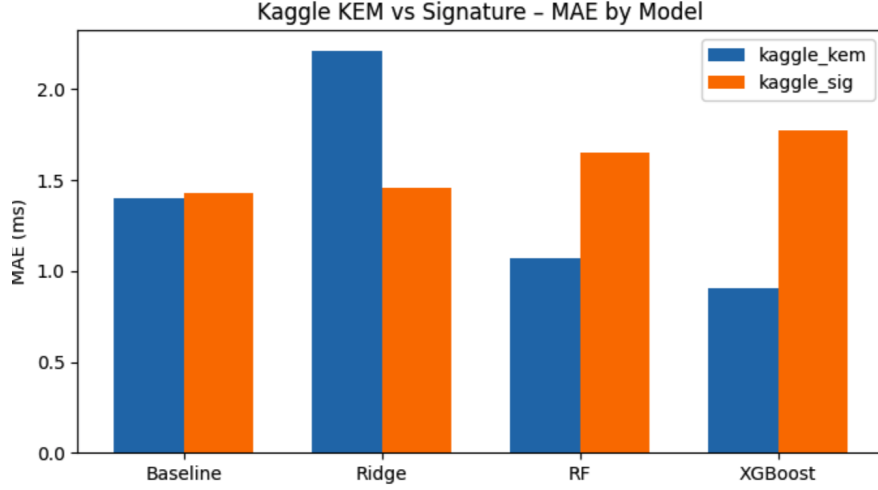


Fig. 3. Mean MAE (on `log_latency`) for Kaggle KEM and signature subsets across models. Signature schemes exhibit higher prediction error than KEMs for all models, indicating that they are inherently harder to model. XGBoost provides the lowest error in both categories.

ples. The held-out machine has somewhat higher median latency than the training machines, but a less extreme upper tail.

5.6.0.b. Evaluation metric: empirical coverage.

For each quantile level α , I checked what fraction of test points actually fell below the predicted α -quantile. If the model is well calibrated, this empirical coverage should be close to the nominal level (for example, about 95% of points should be below the predicted p_{95} curve).

5.6.0.c. Results.

TABLE III
EMPIRICAL COVERAGE OF QUANTILE REGRESSION MODELS FOR LOG-LATENCY.

	Nominal level	Train coverage	Held-out machine coverage
p_{50}	0.50	0.503	0.542
p_{95}	0.95	0.949	1.000
p_{99}	0.99	0.996	1.000

Table III summarizes the empirical coverage for each quantile model. On the training machines, the coverage values are very close to their nominal levels (for example, 0.503 vs. 0.50 for p_{50} and 0.949 vs. 0.95 for p_{95}), which suggests that the quantile regressors are well calibrated and capture both the center and the heavy tail of the latency distribution.

On the held-out (most powerful) machine, the picture changes. The p_{50} model modestly over-covers, with about 54% of observations falling below the predicted median, and the p_{95} and p_{99} models are strongly conservative (coverage of 1.000 at both levels). In other words, the tail predictions for this faster system behave more like upper bounds than tight quantile estimates. This pattern indicates that the models transfer reasonably well in the middle of the distribution, but their extreme-quantile predictions become pessimistic when applied to hardware whose tail latencies are lower than those seen during training. In practice, the p_{95} and p_{99} curves can still be useful as safe latency ceilings for the held-out machine, but they would need additional hardware-aware features or calibration to provide sharp, machine-specific tail estimates.

Table III summarizes the empirical coverage on the training machines and on the held-out machine.

5.7. Random Forest Bias–Variance Tradeoff

Model	MAE _{train}	RMSE _{train}	MAE _{test}	RMSE _{test}
RF (unconstrained)	0.0135	0.0561	0.5901	0.7917
RF (max_depth=15, min_samples_leaf=2)	0.3058	0.5031	0.7473	0.8802
RF (max_depth=20, min_samples_leaf=2)	0.1992	0.3572	0.6493	0.8308
RF (max_depth=10, min_samples_leaf=5)	0.4804	0.7236	0.9364	1.0595

TABLE IV
TRAIN AND TEST PERFORMANCE FOR RANDOM FOREST CONFIGURATIONS, ILLUSTRATING THE BIAS–VARIANCE TRADEOFF.

To study how model complexity affects performance, I trained several Random Forest configurations that share the same preprocessing pipeline but differ in maximum tree depth and minimum samples per leaf. Table IV reports training and test MAE/RMSE on `log_latency` for these settings.

5.7.0.a. Unconstrained Random Forest.

An initial configuration with effectively unbounded tree depth achieves very low error on the training data (MAE 0.0135, RMSE 0.0561), but substantially higher error on the test set (MAE 0.5901, RMSE 0.7917). The large train–test gap indicates clear overfitting: the model has very low bias on the training set but high variance when predicting new examples.

5.7.0.b. Moderately Regularized Forests.

Imposing a maximum depth of 15 and a minimum of two samples per leaf reduces overfitting. This configuration yields a training MAE of 0.3058 and RMSE of 0.5031, and a test MAE of 0.7473 and RMSE of 0.8802. Increasing the maximum depth to 20 further improves both training and test performance, with MAE 0.1992 and RMSE 0.3572 on train, and MAE 0.6493 and RMSE 0.8308 on test. The depth-20 forest provides the best generalization performance among the Random Forest settings evaluated.

5.7.0.c. Strongly Regularized Forest.

A more aggressively regularized model with `max_depth = 10` and `min_samples_leaf = 5` produces higher training and test errors (train MAE 0.4804, test MAE 0.9364) than the deeper forests, even though the train–test gap is somewhat smaller. In this problem, the extra bias introduced by shallow trees and larger leaves outweighs the variance reduction, leading to worse overall accuracy.

5.7.0.d. Preferred Configuration.

Overall, these experiments illustrate the classic bias–variance tradeoff in tree-based ensembles. Fully unconstrained forests overfit the training data, while overly shallow forests underfit. An intermediate setting with `max_depth = 20` and `min_samples_leaf = 2` strikes a more favorable balance, achieving the best test MAE and RMSE among the Random Forest configurations and serving as the preferred Random Forest model in the comparisons with Ridge and XGBoost.

5.8. XGBoost Modeling and Tuning

Gradient-boosted decision tree models using the XGBoost library were also used, first with a reasonably expressive base configuration and then with small manual adjustments. The main XGBoost model used 600 trees with maximum depth 6 and a learning rate of 0.05, together with stochastic subsampling of rows and columns (`subsample = 0.8`, `colsample_bytree = 0.8`) and

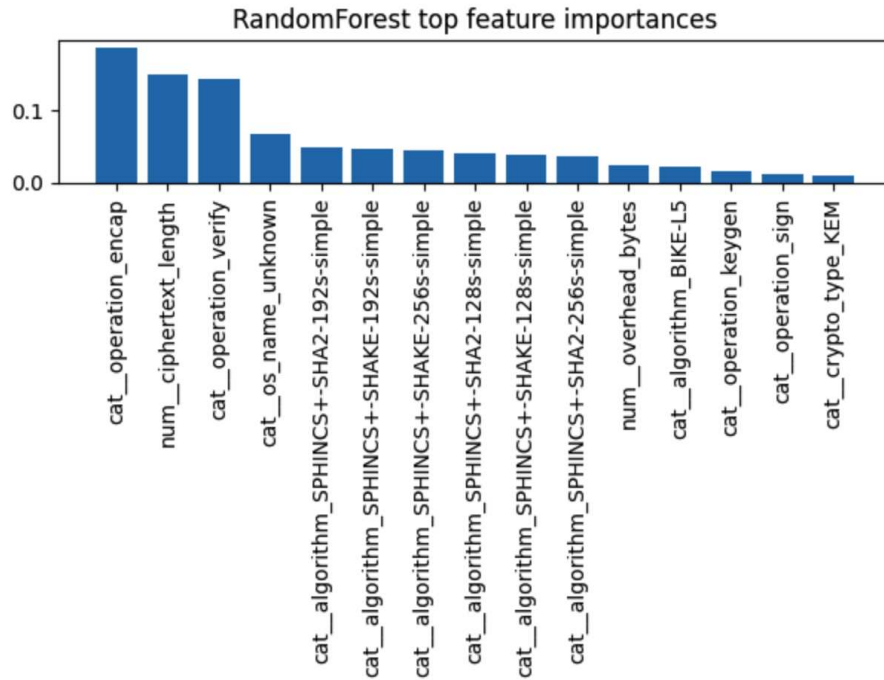


Fig. 4. Top feature importances for the preferred Random Forest configuration. Operation type (especially encapsulation), ciphertext length, operating system, and specific algorithm variants are among the most influential predictors of `log_latency`, highlighting the importance of both algorithm-level and system-level features.

the default L2 regularization (`reg_lambda = 1.0`). This configuration achieved the best overall performance on the held-out machine among all models I considered (see Table ??), improving both MAE and RMSE relative to the Random Forest and the baseline predictor.

To explore regularization effects, I experimented with adding L1 penalty by setting `reg_alpha = 0.1`. As shown in Table ??, this variant slightly increased both training and test errors and therefore did not improve generalization. I also tested a more conservative configuration with shallower trees and fewer boosting rounds, which reduced training accuracy and led to worse performance on the held-out machine, indicating underfitting. Based on these experiments, I selected the base XGBoost configuration as the final model.

6. Conclusions

This project used simple regression models to explore how post-quantum cryptographic latency depends on algorithm choice, input characteristics, and hardware. By combining a public PQC benchmark dataset with measurements from a custom `liboqs`-based harness, I trained and compared baseline linear models, Random Forests, and XGBoost on a unified feature set and evaluation pipeline.

Overall, the results show that:

- Nonlinear tree-based models (Random Forest and especially XGBoost) substantially outperform a regularized linear baseline in predicting PQC latency on the `log_latency` scale.
- System heterogeneity is a major source of variation and prediction difficulty: per-system errors vary noticeably, and models trained on one set of machines degrade when evaluated on held-out hardware.
- Generalization to unseen systems remains challenging; even the best model shows a clear

performance drop under leave-one-system-out splits, suggesting that some form of hardware-aware modeling or calibration is needed.

- Tail errors are significantly larger than mean errors, and quantile regression results confirm that rare high-latency cases are systematically harder to predict than typical behavior.

Taken together, these findings indicate that straightforward statistical learning models can capture much of the structure in PQC performance data and provide useful estimates of typical latency across algorithms and systems. At the same time, system-specific effects and worst-case behavior remain difficult to model accurately, which is important for teams planning PQC deployments in latency-sensitive applications.

7. Future Work

This project took a first pass at predicting PQC latency with relatively simple models and a limited feature set. There are several natural extensions that I did not explore in this work:

- **Apply more systematic XGBoost tuning** using group-aware cross-validation and automated hyperparameter search, rather than the small number of manual settings explored here.
- **Model each algorithm family separately.** Instead of training a single model across all KEM and signature schemes, future work could fit separate models for each algorithm family (e.g., ML-KEM, ML-DSA, SPHINCS+). This might reduce heterogeneity and make it easier to capture family-specific timing patterns.
- **Add concurrency-aware features.** The current models treat each operation as if it runs in isolation. Including features that describe concurrent load (for example, number of parallel operations or competing processes) would make the latency predictions more realistic for multi-tenant systems and high-traffic services.
- **Use Bayesian models for uncertainty.** The quantile regression experiments provide rough upper bounds on latency, but a fully Bayesian approach could give richer uncertainty estimates, such as predictive intervals that account for both model and data uncertainty.
- **Benchmark more diverse hardware.** The experiments here are limited to a small set of EC2 instances and two local machines. Extending the benchmark to GPUs, newer CPU generations, and other cloud providers would test how well the modeling approach scales to a wider range of deployment environments.

Acknowledgements

I would like to thank Dr. Vikalo, instructor of ECE 381V (Statistical Machine Learning) at the University of Texas at Austin, and Dongjun Kim, the course teaching assistant, for guidance and feedback on this project. I am also grateful to Rana Tariq for releasing the “PQC Algorithms Benchmark Data” dataset on Kaggle [2], and to the Open Quantum Safe project maintainers for providing the [4] library used in my benchmarking harness. Finally, I acknowledge the use of [5] OpenAI ChatGPT system for assistance with code frameworks, troubleshooting, and editing the manuscript.

References

- [1] B. John, “Predictive analysis of quantum-resistant algorithms using machine learning approaches,” 05 2025.
- [2] R. Tariq, “Pqc algorithms benchmark data,” <https://www.kaggle.com/datasets/ranatariq09/pqc-algorithms-benchmark-data>, 2024, accessed 2025-12-06.
- [3] M. Dewey-Sumner, “381v pqc performance modeling,” <https://github.com/newspeedtech/381V-PQC-Perf-Modeling>, 2024, course project repository, accessed 2025-12-06.
- [4] Open Quantum Safe Project, “liboqs: C library for quantum-safe cryptographic algorithms,” <https://github.com/open-quantum-safe/liboqs>, 2024, accessed: 2025-12-07.
- [5] OpenAI, “ChatGPT: Large language model,” <https://chat.openai.com>, 2024, model: GPT-5.1 Thinking, Accessed: 2025-12-07.